

Supplementary Material for “OH-MAS: Contracting the Search Space for Static-Analysis Warning Repair”

This supplementary material extends the main paper with full implementation details, architectural configurations, experimental results, and complete prompt templates. It is provided to support reviewer inspection.

APPENDIX A OH-MAS IMPLEMENTATION DETAILS

A. Event-driven Repair State Machine

OH-MAS is implemented as a finite repair loop over typed events rather than as a free-form conversational transcript. The event bus dispatches `task.profiled`, `context.ready`, `patches.ready`, and `audit.done`. A passing audit terminates the run; a failing audit re-enters AO with `PreviousAuditFeedback` and the accumulated `task.append`. Thus retries are not independent restarts: each new round carries the diagnosis and constraints produced by earlier rounds.

TABLE S1
EVENT-STATE MACHINE IMPLEMENTED BY OH-MAS.

Event	Producer	Consumer / effect
<code>task.started</code>	Orchestrator	Initialize trace and retry state.
<code>task.profiled</code>	AO	GW builds context and contract.
<code>context.ready</code>	GW	CP generates candidate patches.
<code>patches.ready</code>	CP	DA audits candidates.
<code>audit.done/pass</code>	DA	Terminate with accepted patch.
<code>audit.done/fail</code>	DA	Retry with typed feedback if budget remains.

The default experiment budget permits two retries, yielding at most three attempts: Easy, Medium, and Hard. The orchestrator stores the current retry index, current profile, context, patch map, last failed audit, and the accumulated task append, making the state machine replayable from trace files.

B. Core Data Schemas

Table S2 summarizes the schemas used to pass typed information between agents. The key object is `RepairContract`, a three-field structure that carries the bounded edit space from GW to CP.

C. Workspace Isolation and Tracing

Each instance is run in an isolated workspace cloned from the source repository and checked out at the target commit. The cleanup path handles Docker-owned files by attempting permission repair and renaming uncleaned workspaces out of the way. Traces are JSONL records with sequence number, UTC timestamp, event kind, and payload. Agent calls, published events, GW summaries, CP worker trajectories, patches, and audit verdicts are co-located under the task trace directory.

TABLE S2
CORE OH-MAS SCHEMAS.

Schema	Key fields	Role
<code>Alarm</code>	file, rule, line span, message	Linter warning input.
<code>RepairContract</code>	<code>must_fix</code> , <code>must_not_touch</code> , <code>allowed_transformations</code>	Bounded edit space.
<code>KnowledgePack</code>	<code>rule_templates</code> , <code>experiences</code>	OH-KB content for GW.
<code>TaskProfiledEvent</code>	mode, GW input, CP input	AO decision payload.
<code>PatchDiagnostic</code>	<code>failed_level</code> , reason, violations	Per-patch audit evidence.
<code>PreviousAuditFeedback</code>	<code>failed_level</code> , <code>failed_patches</code> , diagnostics	Retry signal.

APPENDIX B REPAIR CONTRACT CONSTRUCTION

A. Contract Fields

A repair contract is a triple $\mathcal{C}_w = \langle F, N, A \rangle$, where $F = \text{MUST_FIX}$, $N = \text{MUST_NOT_TOUCH}$, and $A = \text{ALLOWED_TRANSFORMATIONS}$. `MUST\_FIX` lists the regions that must be inspected or edited. `MUST\_NOT\_TOUCH` lists invariants that should be preserved, such as public signatures, exported ArkTS structures, decorators, ownership-sensitive C++ definitions, and lifecycle obligations. `ALLOWED\_TRANSFORMATIONS` contains rule-specific repair patterns synthesized from graph evidence and rule templates.

B. Mode-dependent Graph Slicing

GW builds the contract from the warning span, repository graph, rule templates, and previous audit evidence. Easy mode starts from the local region. Medium mode adds nearby data/control dependencies and richer framework invariants. Hard mode admits a broader cross-file slice when earlier constraints indicate that the valid repair lies outside the local region. Escalation may enlarge the slice, but accumulated `MUST\_NOT\_TOUCH` constraints are retained.

Prompt B.1: GW tool-use protocol for contract synthesis

```

1 GW is an LLM-driven tool-use agent. The model
  iteratively calls the following tools until it emits
  \texttt{finalize\_context}; the trace records every
  call.
2
3 \textbf{Phase 1 --- Evidence collection (2--3 steps)}
4 \begin{enumerate}[leftmargin=1.4em,itemsep=1pt,
  topsep=1pt]
5 \item \texttt{show\_alarm} to read the target
  warning.
6 \item \texttt{show\_knowledge} to load rule
  templates and prior repair experiences.
7 \item \texttt{read\_lines} / \texttt{read\_symbol} /
  \texttt{grep\_file} on the alarm span and adjacent
  dependencies, optionally preceded by \texttt{show\_
  graph\_overview} / \texttt{list\_neighbors} when
  the mode is \emph{medium} or \emph{hard}.

```

TABLE S3
EXAMPLE REPAIR CONTRACT SYNTHESIZED BY GW FOR INSTANCE OH_0186 (RULE
@PERFORMANCE/HP-ARKUI-NO-STATE-VAR-ACCESS-IN-LOOP, MODE = MEDIUM). THE ALARM FILE IS
PERMISSIONMANAGER/SRC/MAIN/ETS/PERMISSIONSHEET/PERMISSIONSTATESHEETDIALOG.ETS.

Field	Item	Concrete content
MUST_FIX	#1	PermissionStateSheetDialog.ets:170 — for loop reads @State this.reqUserPermissions; extract to local const before the loop.
	#2	:338 — for loop writes @State this.groupName inside nested loop at line 328; hoist write outside the loop.
	#3	:341 — for loop writes @State this.isGranted inside loop at line 328; move assignment outside.
	#4	:342 — for loop writes @State this.folderStatus inside loop at line 328; move assignment outside.
	#5	:345 — for loop reads @State this.folderStatus inside loop at line 328; extract to local const first.
	#6	:576 — nested loops (567, 568) write @State this.reason; accumulate in local, assign once after loops.
	#7	:653 — loop at line 646 reads @State this.accurateIsOn; extract to local const before the loop.
	#8	:655 — loop at line 646 writes @State this.accurateIsOn; use local variable, assign once after the loop.
MUST_NOT_TOUCH	#1	PermissionStateSheetDialog.ets:build() — never extract @State variables inside build(), which breaks ArkUI reactivity.
	#2	PermissionStateSheetDialog.ets:ForEach — never extract @State variables consumed by ForEach inside build(); they must stay reactive for UI updates.
ALLOWED_TRANSFORMATIONS	#1	Read @State once into a local const before procedural for/while loops.
	#2	Write @State after the loop completes; collect intermediate values in a local variable inside the loop body.
	#3	For nested loops writing @State: accumulate in a local during iteration, assign to @State once outside all loops.
	#4	Move @State assignments that do not depend on loop iteration outside the loop body.
	#5	For @State reads used in loop conditions: extract to a local const before the loop starts.

```

8 \end{enumerate}
9
10 \textbf{Phase 2 --- Contract writing (1--3 steps):}
11 \begin{enumerate}[leftmargin=1.4em,itemsep=1pt,
12 topsep=1pt]
13 \item \texttt{set\_must\_fix}(items=[``<file>:<line>
14 --- <pattern and why>'', \ldots])).
15 \item \texttt{set\_must\_not\_touch}(items=[``<file
16 >:<symbol> --- <why>'', \ldots])).
17 \item \texttt{set\_allowed\_transformations}(items
18 =[[``<concrete pattern from templates / experiences
19 >'', \ldots])).
20 \end{enumerate}
21
22 \textbf{Phase 3 --- Finalization (1 step):}
23 \begin{enumerate}[leftmargin=1.4em,itemsep=1pt,
24 topsep=1pt]
25 \item \texttt{finalize\_context}(reasoning,
26 confidence)} to emit the typed \texttt{RepairContract} that CP consumes.
27 \end{enumerate}
28
29 \textbf{Negative requirements:}
30 \begin{itemize}[leftmargin=1.4em,itemsep=1pt,topsep=
31 1pt]
32 \item Do not generate a patch; GW only writes the
33 contract.
34 \item Do not include unrelated files simply because
35 they are textually similar.
36 \item Do not omit framework invariants for ArkUI
37 decorators, lifecycle methods, or state bindings.
38 \item On retry, must\_fix must address both the
39 original alarm and any newly introduced rules from
40 the previous attempt.
41 \end{itemize}

```

The tool sequence above is recorded verbatim in every GW trajectory; the trace file contains the tool name, arguments, and observation for each step.

APPENDIX C TYPED DIAGNOSTIC FEEDBACK

A. Three-tier Audit

DA uses a hierarchical stop-at-first-failure policy. L1 checks whether a candidate is a valid and applicable unified diff. L2 checks whether the target warning is removed and whether the linter reports syntax or parse errors. L3 checks whether the post-patch repository warning set introduces warnings absent from the baseline snapshot.

Listing 1. Three-tier audit procedure.

```

Audit(candidate_patch, alarm):
    baseline = collect_repo_linter_snapshot()
    if not apply_unified_diff(candidate_patch): return Fail(
        L1)
    post = collect_repo_linter_snapshot()
    if syntax_or_parse_error(post): return Fail(L2)
    if target_warning_still_present(post, alarm): return Fail(
        L2)
    if new_warning_exists(post - baseline): return Fail(L3)
    return Pass(candidate_patch)

```

B. Failure-to-Constraint Mapping

C. Monotonic Constraint Accumulation

The retry prompt contains a persistent task append. When a previous audit exists, AO uses either LLM-produced mandatory constraints or a deterministic fallback template, and then appends previously accumulated constraints. Thus a later round cannot forget that an earlier patch failed because a region was edited incorrectly or because an introduced warning had to be avoided.

TABLE S4
TYPED FAILURES AND THEIR RETRY CONSTRAINTS.

Failure	Constraint injected into the next attempt
L1	Re-read target files; use exact current text for edits; do not reuse stale line ranges.
L2	Repair residual warning locations; target the exact linter-reported pattern.
L3	Preserve or repair introduced warning locations; add regression context as MUST_NOT_TOUCH or required repair.

APPENDIX D MODEL ROUTING AND WORKER EXECUTION

A. Routing Policy

AO treats the model pool as a configurable candidate set rather than as a fixed per-mode table. The experiment configuration declares the full pool for each run, and AO selects from it using warning language, rule family, retry index, and failure tier.

TABLE S5
MODEL POOLS USED BY RQ3. AO ROUTES WITHIN EACH FULL POOL AT RUNTIME.

Pool	Purpose	Complete CP model pool
P1	Balanced	Claude Sonnet 4.5; Gemini 2.5 Flash; Kimi K2.5
P2	Cost-effective	Kimi K2.5; Gemini 2.5 Flash; MiniMax M2.5
P3	Frontier	Claude Opus 4.5; Claude Sonnet 4.5; Gemini 2.5 Flash

B. Worker-visible Context

CP workers do not receive the entire knowledge base or repository as raw input. AO packages retry constraints into `prompt_injection.task_append`; GW converts graphs, templates, and experiences into a unified contract; and CP receives a worker-visible projection containing the target alarm, GW context, and AO task append.

C. Parallel Candidate Generation

In multi-model settings, CP launches up to three workers concurrently. Each worker has the same alarm and contract but a different model backend. The first passing patch is accepted; otherwise diagnostics are accumulated for the next retry.

APPENDIX E COMPLETE PROMPT INVENTORY

Runtime values such as concrete paths, snippets, graph slices, and linter messages are shown with angle-bracket placeholders. PromptBox is used only for prompts.

A. Adaptive Orchestrator Prompts

Prompt E.1: AO strategy decision prompt

```
1 You are the Adaptive Orchestrator in OH-MAS.
2 Given the target warning, retry index, current mode,
3 previous audit feedback,
4 and the allowed model registry, decide how the next
5 repair attempt should run.
```

```
5 Inputs:
6 - target alarm: <file, line span, rule id, message,
  language>
7 - retry index: <0 | 1 | 2>
8 - previous audit feedback: <none | L1 | L2 | L3
  diagnostics>
9 - allowed CP models: <model registry>
10
11 Return a JSON object with:
12 {
13   "gw_input": {"build_semantic_graph": <true | false
14   >,
15               "extract_constraints": <true | false
16   >},
17   "cp_strategy": {"models": [<selected model ids>],
18                  "model_count": <integer>,
19                  "task_append": "<MANDATORY
20   CONSTRAINTS>"},
21   "rationale": "<brief explanation of decisions>"
22 }
23
24 Rules:
25 - Do not expose raw KB items directly to CP; GW must
26   synthesize them into a contract.
27 - L1 feedback focuses on applicability and exact
28   file text.
29 - L2 feedback includes residual linter locations and
30   still-violating patterns.
31 - L3 feedback includes introduced warnings and
32   invariants to preserve.
33 - Preserve constraints accumulated from earlier
34   retries.
```

B. Graph Weaver Prompts

Prompt E.2: GW repair-contract synthesis prompt

```
1 You are the Graph Weaver in OH-MAS. Your task is not
2 to edit code.
3 Your task is to construct a Repair Contract that
4 bounds the Constrained Patcher.
5
6 You must identify:
7 1. must_fix locations;
8 2. must_not_touch invariants;
9 3. allowed_transformations.
10
11 Use the alarm, dependency graph slice, rule
12 templates, repair experiences,
13 and previous audit feedback. If retry feedback
14 exists, make the contract more
15 specific to the failed tier. Finalize the context
16 only after all three contract
17 fields are populated or explicitly justified as
18 empty.
```

Prompt E.3: GW tool-use workflow prompt (summary)

```
1 The full GW tool-use protocol is given in Prompt~\
2 ref{prompt:gw-tools-protocol} (Appendix~\ref{app:
3 contract}). Briefly:
4
5 \begin{enumerate}[leftmargin=1.4em,itemsep=1pt,
6 topsep=1pt]
7 \item \texttt{show\_alarm} $\rightarrow$ \texttt{read\_lines}
8 / \texttt{read\_symbol} / \texttt{grep\_file} on the
9 alarm span and adjacent dependencies.
10 \item \texttt{set\_must\_fix}, \texttt{set\_must\_
11 not\_touch}, \texttt{set\_allowed\_transformations}
12 with concrete items.
13 \item \texttt{finalize\_context} (reasoning,
14 confidence) to emit the typed \texttt{
15 RepairContract}.
16 \end{enumerate}
17
18 \textbf{Negative requirements:}
19 \begin{itemize}[leftmargin=1.4em,itemsep=1pt,topsep=
20 1pt]
21 \item Do not generate a patch.
```

```

12 \item Do not include unrelated files simply because
    they are textually similar.
13 \item Do not omit framework invariants for ArkUI
    decorators, lifecycle methods, or state bindings.
14 \item On retry, must\_fix must address both the
    original alarm and any newly introduced rules from
    the previous attempt.
15 \end{itemize}

```

C. Constrained Patcher Prompts

Prompt E.4: CP target-alarm section

```

1 # Target Alarm
2
3 **File:** `<alarm.file>`
4 **Rule:** `<alarm.rule>`
5 **Location:** Lines <alarm.line_start>-<alarm.
    line_end>
6 **Message:** <alarm.message>

```

Prompt E.5: CP graph-context section (repair contract)

```

1 # Repair Contract from GW Agent
2
3 ## Must Fix
4 These locations/patterns MUST be addressed to
    eliminate the alarm(s):
5
6 - <must\_fix\_item\_1>
7 - <must\_fix\_item\_2>
8 - <must\_fix\_item\_3>
9 ...
10
11 ## Must Not Touch
12 These elements MUST NOT be modified to avoid
    regressions:
13
14 - <must\_not\_touch\_item\_1>
15 - <must\_not\_touch\_item\_2>
16 ...
17
18 ## Allowed Transformations
19 Apply ONE of these concrete transformation patterns:
20
21 - <allowed\_transformation\_1>
22 - <allowed\_transformation\_2>
23 - <allowed\_transformation\_3>
24 ...
25
26 ## GW Derivation Rationale
27 <the GW LLM's free-text rationale explaining why
    these constraints were chosen>

```

Prompt E.6: CP additional instructions from AO

```

1 ## Additional Instructions from AO
2
3 <raw task\_append string produced by AO; contains
    typed mandatory constraints
4 and accumulated constraints from earlier retries>

```

D. Retry-specific Prompt Fragments

Prompt E.7: CP L1 retry protocol

```

1 =====
2 ===  RETRY EXECUTION PROTOCOL  ===
3 =====
4
5 **Retry Attempt:** \#<retry\_index>
6 **Previous Failure:** <failed\_level> -- <reason>
7
8 ## L1 Failure Recovery Workflow
9
10 Your previous patch failed to apply. Follow this
    workflow:
11

```

```

12 1. \textbf{Review AO's MANDATORY CONSTRAINTS above}
    for exact failure locations
13 2. \textbf{Re-read ALL target files} using \texttt{
    read\_file <path>}
14    - Files may have changed since your last attempt
15    - DO NOT reuse old\_str from memory
16 3. \textbf{Copy exact strings} from current file
    content for old\_str
17 4. \textbf{Verify uniqueness} of old\_str before
    calling edit\_file
18
19 ## Universal Execution Rules
20
21 - Always \texttt{read\_file} before \texttt{edit\_
    file} (never trust memory)
22 - Copy old\_str from CURRENT file content (not
    previous attempts)
23 - Ensure old\_str is unique in the file
24 - State your repair plan in one sentence before
    editing
25
26 =====

```

Prompt E.8: CP L2 retry protocol

```

1 =====
2 ===  RETRY EXECUTION PROTOCOL  ===
3 =====
4
5 **Retry Attempt:** \#<retry\_index>
6 **Previous Failure:** <failed\_level> -- <reason>
7
8 ## L2 Failure Recovery Workflow
9
10 The target alarm still exists. AO has provided root
    cause analysis above.
11
12 1. \textbf{Study AO's MANDATORY CONSTRAINTS section
    above} carefully
13    - Note EXACT line numbers where linter detected
    violations
14    - Review code snippets showing problematic
    patterns
15    - Understand AO's analysis of WHY previous
    patches failed
16 2. \textbf{Read target file} with \texttt{read\_file
    } to see current state
17 3. \textbf{Plan your fix} targeting the SPECIFIC
    patterns analyzed by AO
18 4. \textbf{Execute edit\_file} for those exact
    locations
19
20 ## Universal Execution Rules
21
22 - Always \texttt{read\_file} before \texttt{edit\_
    file} (never trust memory)
23 - Copy old\_str from CURRENT file content (not
    previous attempts)
24 - Ensure old\_str is unique in the file
25 - State your repair plan in one sentence before
    editing
26
27 =====

```

Prompt E.9: CP L3 retry protocol

```

1 =====
2 ===  RETRY EXECUTION PROTOCOL  ===
3 =====
4
5 **Retry Attempt:** \#<retry\_index>
6 **Previous Failure:** <failed\_level> -- <reason>
7
8 ## L3 Failure Recovery Workflow
9
10 Your previous patch introduced new warnings. AO has
    analyzed them above.
11

```

```

12 1. \textbf{Read AO's MANDATORY CONSTRAINTS section
13   above}
14   - Note ALL introduced warning rules and locations
15   - Review AO's root cause analysis for each
16   violation
17   - Understand WHY your previous patch triggered
18   these warnings
19 2. \textbf{Read EVERY file mentioned} in AO's
20   analysis
21   - Use exact paths from the violations list
22   - If path fails, use \texttt{search\_code <
23   basename>} to locate it
24 3. \textbf{Consult the repair contract} for
25   transformation guidance
26   on the introduced rules
27   - GW has synthesized allowed\_transformations
28   from repair experiences
29   - Study the fix patterns in the Allowed
30   Transformations section
31 4. \textbf{Address EACH introduced warning} based on
32   :
33   - AO's root cause analysis (WHY it was triggered)
34   - Knowledge pack patterns (HOW to fix it)
35 5. \textbf{Verify holistically} before submit\_patch
36   :
37   - Original alarm fixed
38   - No warnings from AO's list remain
39   - No new syntax errors introduced
40
41 ## Universal Execution Rules
42
43 - Always \texttt{read\_file} before \texttt{edit\_
44   _file} (never trust memory)
45 - Copy old\_str from CURRENT file content (not
46   previous attempts)
47 - Ensure old\_str is unique in the file
48 - State your repair plan in one sentence before
49   editing
50
51 =====

```

Prompt E.10: CP recommended workflow prompt

```

1 # Recommended Workflow
2
3 \begin{verbatim}
4 # Step 1: Inspect the alarm location
5 read_file <alarm_file>
6
7 # Step 2-(step_limit-3): Read other relevant files
8 from context (optional)
9 # read_file <other_file>
10 # search_code <pattern> # If you need to find
11 related code
12
13 # Step (step_limit-2): Make your edits (can call
14 multiple times)
15 edit_file {"path": "...", "old_str": "...", "new_str": "..."}
16
17 # Step (step_limit-1): Additional edits if needed (
18 multi-file patches)
19 # edit_file {"path": "...", "old_str": "...", "
20 new_str": "..."}
21
22 # Step step_limit: Submit the patch
23 submit_patch
24 \end{verbatim}
25
26 \textbf{Strategy tips:}
27 - Reserve at least 2 steps for editing + submission
28 - Read files before editing to get current content
29 - Use snippets from context as hints, but always
30 verify with read\_file
31 - Multiple edit\_file calls are allowed for complex
32 repairs

```

APPENDIX F EXPERIMENT CONFIGURATIONS

A. P1/P2/P3 Model-pool Configurations

Table S5 lists the complete model pool exposed to CP in each RQ3 configuration. Full YAML files are included in the artifact; this appendix reports the relevant fields only.

Listing 2. P1 CP model pool.

```

cp.models_registry:
  - openrouter/anthropic/claude-sonnet-4.5
  - openrouter/google/gemini-2.5-flash
  - openrouter/moonshotai/kimi-k2.5
runtime.max_retries: 2
cp.max_parallel_workers: 3

```

Listing 3. P2 CP model pool.

```

cp.models_registry:
  - openrouter/moonshotai/kimi-k2.5
  - openrouter/google/gemini-2.5-flash
  - openrouter/minimax/minimax-m2.5
runtime.max_retries: 2
cp.max_parallel_workers: 3

```

Listing 4. P3 CP model pool.

```

cp.models_registry:
  - openrouter/anthropic/claude-opus-4.5
  - openrouter/anthropic/claude-sonnet-4.5
  - openrouter/google/gemini-2.5-flash
runtime.max_retries: 2
cp.max_parallel_workers: 3

```

B. Ablation Configuration Files

Listing 5. Key configuration deltas for A2' and A3'.

```

# A2': retry preserved, typed accumulation removed
ao.feedback_mode: free_text_nonaccumulative
runtime.max_retries: 2

# A3': graph slicing preserved, offline templates removed
oh_kb.provider: graph_only
ao.enable_synthetic_knowledge: false
gw.enable_rule_protocols: false
gw.enable_rule_mode: false
cp.enable_rule_guide: false

```

APPENDIX G ADDITIONAL EXPERIMENTAL RESULTS

A. Full P1/P2/P3 Results

B. Additional Ablation Reports

APPENDIX H ABLATION SUBSETS AND ABLATION IMPLEMENTATION

A. Ablation Subset Instances

The ablation subset contains 100 ArkTS and 100 C/C++ instances sampled with seed 42 by proportional quota sampling over rule_id. Each rule receives a quota $q_r = \max(1, \text{round}(|D_r|/|D| \cdot 100))$, quotas are adjusted by descending group size until the total is 100, instances are drawn without replacement from each rule group, and the selection is sorted by instance_id. The full per-instance fields (project, target file, warning message, commit hash, buggy snippet) are archived in the artifact. Tables S9–S10 list every selected identifier.

TABLE S6
CONFIGURATION DIFFERENCES FOR ABLATION EXPERIMENTS.

Variant	Removed mechanism	Retry budget	Knowledge / GW	CP pool
A1	Model pool	2	Graph + templates enabled	One model, one worker
A2	Adaptive typed retry	0	Graph + templates enabled	P1 pool
A2'	Typed cumulative feedback	2	Graph + templates enabled	P1 pool; free-text non-cumulative feedback
A3	Repair contract	2	oh_kb=null; GW LLM disabled	P1 pool
A3'	Offline templates	2	graph_only; rule protocols/templates disabled	P1 pool

TABLE S7
DETAILED PER-LANGUAGE RESULTS FOR P1/P2/P3. C/C++ SPR FOLLOWS THE FINAL EVALUATION PROTOCOL USED IN TABLE ??; OTHER C/C++ DIAGNOSTICS ARE FROM THE DETAILED EXECUTION REPORTS.

Pool	Language	<i>N</i>	PAR	SPR	Pass@1	Avg. rounds
P1	ArkTS	382	93.5%	81.4%	58.6%	1.36
P1	C/C++	359	96.7%	74.1%	43.2%	1.49
P2	ArkTS	382	95.6%	79.8%	52.6%	1.38
P2	C/C++	359	86.6%	71.3%	42.9%	1.63
P3	ArkTS	382	95.3%	78.3%	57.6%	1.35
P3	C/C++	359	92.8%	74.4%	46.0%	1.45

TABLE S8
DETAILED ABLATION RESULTS ON THE 200-INSTANCE SUBSET.

Variant	Language	<i>N</i>	PAR	SPR	Pass@1
A1-Claude	ArkTS	100	95%	78%	60%
A1-Claude	C/C++	100	95%	75%	41%
A1-Gemini	ArkTS	100	98%	83%	62%
A1-Gemini	C/C++	100	96%	68%	43%
A1-Kimi	ArkTS	100	83%	73%	50%
A1-Kimi	C/C++	100	96%	72%	44%
A2	ArkTS	100	99%	74%	74%
A2	C/C++	100	96%	52%	52%
A2'	ArkTS	100	99%	78%	69%
A2'	C/C++	100	96%	63%	48%
A3	ArkTS	100	99%	83%	60%
A3	C/C++	100	96%	58%	48%
A3'	ArkTS	100	97%	73%	56%
A3'	C/C++	100	93%	69%	40%

B. Implementation of Each Ablation Variant

The ablations are implemented as configuration-level mechanism removals, not as post-hoc filtering. A1 restricts CP to one worker/model. A2 removes adaptive retry by setting the retry budget to zero. A2' preserves retry and the model pool but changes AO feedback to free-form non-cumulative text. A3 disables both KB and GW LLM contract synthesis. A3' preserves graph-only slicing but removes offline templates, synthetic knowledge, rule protocols, rule-mode guidance, and CP static rule guides.

APPENDIX I MANUAL ORACLE-VERDICT REVIEW

A. Reviewed Instances

The manual-review subset is 50 ArkTS and 50 C/C++ instances drawn with seed 42 by simple random sampling without replacement, independently of the ablation subset. DA

accepted 87 of the 100 patches (45 ArkTS, 42 C/C++); those accepted patches form the annotation package in the artifact. The complete identifier lists are archived in the artifact, and reproduced in Tables S11–S12.

B. Review Criteria

A DA-accepted patch is counted as a true positive only if it removes the target warning while preserving the intended local semantics. For ArkTS, reviewers additionally checked ArkUI lifecycle obligations, decorator semantics, and state rebinding. For C/C++, reviewers checked ownership, API contracts, and whether the patch merely suppresses the warning.

C. Complete Review Results

The five false positives fall into two categories: ArkTS patches that satisfy CodeLinter while missing subtle lifecycle or state-refresh semantics, and one C/C++ patch that satisfies Cppcheck while changing a local semantic contract. This is

TABLE S9
ABLATION SUBSET: ALL 100 ARKTS INSTANCE IDENTIFIERS.

#1–#25	#26–#50	#51–#75	#76–#100
OH_0001	OH_0099	OH_0175	OH_0307
OH_0003	OH_0100	OH_0182	OH_0311
OH_0008	OH_0102	OH_0183	OH_0312
OH_0009	OH_0103	OH_0186	OH_0316
OH_0012	OH_0107	OH_0188	OH_0317
OH_0014	OH_0108	OH_0189	OH_0319
OH_0018	OH_0113	OH_0193	OH_0320
OH_0019	OH_0115	OH_0197	OH_0321
OH_0023	OH_0116	OH_0201	OH_0325
OH_0030	OH_0122	OH_0215	OH_0326
OH_0032	OH_0128	OH_0223	OH_0327
OH_0037	OH_0129	OH_0239	OH_0329
OH_0041	OH_0130	OH_0241	OH_0334
OH_0044	OH_0133	OH_0249	OH_0340
OH_0046	OH_0134	OH_0254	OH_0341
OH_0048	OH_0138	OH_0255	OH_0349
OH_0057	OH_0140	OH_0257	OH_0352
OH_0060	OH_0142	OH_0260	OH_0353
OH_0061	OH_0144	OH_0261	OH_0360
OH_0062	OH_0153	OH_0264	OH_0361
OH_0078	OH_0154	OH_0268	OH_0366
OH_0080	OH_0156	OH_0269	OH_0369
OH_0082	OH_0159	OH_0279	OH_0376
OH_0083	OH_0165	OH_0290	OH_0380
OH_0090	OH_0169	OH_0300	OH_0381

the basis for the 5.7% false-positive-rate bound discussed in the threats-to-validity section.

D. False-Positive Case Details

TABLE S10
ABLATION SUBSET: ALL 100 C/C++ INSTANCE IDENTIFIERS.

#1–#25	#26–#50	#51–#75	#76–#100
CPP_0006	CPP_0092	CPP_0192	CPP_0257
CPP_0009	CPP_0093	CPP_0193	CPP_0266
CPP_0010	CPP_0095	CPP_0197	CPP_0270
CPP_0011	CPP_0097	CPP_0200	CPP_0274
CPP_0014	CPP_0100	CPP_0204	CPP_0279
CPP_0020	CPP_0109	CPP_0206	CPP_0282
CPP_0023	CPP_0111	CPP_0210	CPP_0283
CPP_0024	CPP_0115	CPP_0212	CPP_0287
CPP_0029	CPP_0118	CPP_0216	CPP_0293
CPP_0038	CPP_0119	CPP_0217	CPP_0303
CPP_0041	CPP_0121	CPP_0220	CPP_0306
CPP_0046	CPP_0125	CPP_0221	CPP_0310
CPP_0047	CPP_0129	CPP_0222	CPP_0319
CPP_0048	CPP_0130	CPP_0226	CPP_0324
CPP_0058	CPP_0139	CPP_0232	CPP_0325
CPP_0059	CPP_0143	CPP_0235	CPP_0327
CPP_0065	CPP_0147	CPP_0236	CPP_0328
CPP_0070	CPP_0150	CPP_0239	CPP_0329
CPP_0071	CPP_0155	CPP_0241	CPP_0331
CPP_0074	CPP_0157	CPP_0242	CPP_0332
CPP_0082	CPP_0176	CPP_0243	CPP_0335
CPP_0084	CPP_0178	CPP_0246	CPP_0340
CPP_0085	CPP_0183	CPP_0250	CPP_0343
CPP_0089	CPP_0188	CPP_0255	CPP_0350
CPP_0091	CPP_0190	CPP_0256	CPP_0357

TABLE S11
MANUAL-REVIEW SUBSET: ALL 50 ARKTS INSTANCE IDENTIFIERS.

#1–#25	#26–#50
OH_0004	OH_0175
OH_0013	OH_0177
OH_0014	OH_0184
OH_0016	OH_0195
OH_0017	OH_0215
OH_0023	OH_0217
OH_0045	OH_0230
OH_0048	OH_0236
OH_0050	OH_0259
OH_0053	OH_0275
OH_0058	OH_0280
OH_0072	OH_0288
OH_0080	OH_0302
OH_0082	OH_0303
OH_0102	OH_0309
OH_0111	OH_0310
OH_0112	OH_0328
OH_0113	OH_0333
OH_0115	OH_0347
OH_0120	OH_0358
OH_0126	OH_0360
OH_0136	OH_0367
OH_0141	OH_0374
OH_0143	OH_0378
OH_0173	OH_0380

TABLE S12
MANUAL-REVIEW SUBSET: ALL 50 C/C++ INSTANCE IDENTIFIERS.

#1-#25	#26-#50
CPP_0017	CPP_0167
CPP_0024	CPP_0182
CPP_0029	CPP_0186
CPP_0034	CPP_0187
CPP_0036	CPP_0190
CPP_0037	CPP_0194
CPP_0041	CPP_0195
CPP_0052	CPP_0206
CPP_0064	CPP_0233
CPP_0084	CPP_0237
CPP_0088	CPP_0274
CPP_0099	CPP_0283
CPP_0108	CPP_0286
CPP_0113	CPP_0296
CPP_0117	CPP_0312
CPP_0118	CPP_0317
CPP_0120	CPP_0322
CPP_0126	CPP_0326
CPP_0137	CPP_0328
CPP_0138	CPP_0332
CPP_0139	CPP_0339
CPP_0143	CPP_0344
CPP_0149	CPP_0350
CPP_0151	CPP_0351
CPP_0162	CPP_0353

TABLE S13
DETAILED BREAKDOWN OF THE FIVE FALSE-POSITIVE PATCHES.

Instance	Lang.	Patch summary	Root-cause analysis	Consensus
OH_0017	ArkTS	Replaced inline <code>ListItem/Stack</code> wrapping with a <code>@Reusable</code> component; changed <code>LazyForEach</code> key & item passing.	(1) The parent page changed from passing <code>item.calllog</code> to passing the full item object <code>{calllog, index, showDivifer}</code> , but the child component still accesses <code>this.item.suffix</code> etc. — those are properties of <code>calllog</code> , not of the top-level wrapper, so all display fields become undefined. (2) The <code>@State</code> decorator on <code>item</code> was removed, changing it to <code>private</code> , which breaks UI reactivity when <code>aboutToReuse()</code> reassigns data in a <code>@Reusable</code> component.	Both agree (FP)
OH_0072	ArkTS	Refactored <code>LazyForEach</code> to a <code>@Builder</code> ; changed <code>@Link mPresenter</code> to <code>@Prop</code> .	The <code>@Builder</code> passes <code>mPresenter: \$mPresenter</code> (the two-way <code>@Link</code> syntax using <code>\$</code>), but the child component now declares <code>@Prop mPresenter</code> (one-way). In ArkUI, the <code>\$</code> prefix is semantically tied to <code>@Link/@ObjectLink</code> ; passing <code>\$</code> to a <code>@Prop</code> target is a type mismatch that causes compilation or runtime errors.	Both agree (FP)
OH_0082	ArkTS	Replaced <code>CustomTextInputContent</code> with native <code>TextInput + Button</code> in the group-rename dialog.	The original <code>onConfirm(value)</code> callback compared the user input value against the original name <code>this.item.group.groupName</code> . The new <code>TextInput.onChange()</code> mutates <code>this.item.group.groupName</code> on every keystroke, so at confirm time the comparison becomes <code>this.item.group.groupName == this.item.group.groupName</code> — a self-comparison that is always true. The rename logic is never reached and the group name can never be changed.	Both agree (FP)
CPP_0326	C/C++	Replaced <code>path.find(X) != string::npos</code> with <code>path.rfind(X, 0) == 0</code> in <code>GetNewPath()</code> .	<code>path.find(ROOT_MEDIA_DIR) != npos</code> originally matched <code>ROOT_MEDIA_DIR</code> anywhere in the path; the new <code>path.rfind(ROOT_MEDIA_DIR, 0) == 0</code> only matches it at the beginning. If <code>ROOT_MEDIA_DIR</code> appears in the middle of a path string, the new code omits the prefix, producing a different (and potentially incorrect) path. While the new semantics may better align with the developer's intent, it is a behavioral change relative to the original code.	Both agree (FP)
OH_0143	ArkTS	Converted opacity/scale-based animation to a <code>TransitionEffect</code> using <code>if(this.show)</code> conditional rendering.	Two <code>TransitionEffect.scale()</code> calls are combined via <code>.combine()</code> with identical 200ms durations: <code>scale(0.7,0.7)</code> and <code>scale(1.03,1.03)</code> . They execute in parallel and conflict; only the second scale takes effect. The intended multi-stage overshoot animation ($0.7 \rightarrow 1.03 \rightarrow 1.0$) is broken. Reviewer A flagged this conflict as FP while Reviewer B initially missed it (TP); resolved to FP after joint re-examination.	Disagreed: A=FP, B=TP $\rightarrow$ FP